

Honest Machines

Adversarial Oracles as a First-Class Artifact in Process Mining Tooling

Sean Chatman

wasm4pm Project — v26.4.28

May 2026

Abstract

Process mining tools have long validated themselves against their own outputs. A discovery algorithm is declared correct when it produces a plausible-looking Petri net; a conformance checker is declared working when it returns a number between zero and one. This paper argues that such self-referential validation is epistemically bankrupt. Drawing on the Van der Aalst constitution — *if the code says it worked but the event log cannot prove a lawful process happened, then it did not work* — and on adversarial testing patterns extracted from three independent codebases (dteam, unibit, bcinr), we present a systematic methodology for building process mining tools that can be falsified. We instantiate this methodology in `wasm4pm`, a Rust/WASM process mining platform, producing 65 new tests organized by oracle rank, covering streaming equivalence, ground-truth corpus validation, gate rejection, prediction anti-leakage, OCEL many-to-many semantics, and enterprise dirty data impact. The central contribution is not the tests themselves but the epistemological position they enforce: **a tool that cannot document its own failure modes is not a scientific instrument.**

Contents

1	Introduction: The Problem of the Self-Validating Tool	2
1.1	Scope	2
2	Background	2
2.1	Van der Aalst’s Constitution	2
2.2	Oracle Hierarchy	3
2.3	The PDC 2025 Corpus Pattern	3
3	Methodology: The Adversarial Test Architecture	3
3.1	Anti-Lie Infrastructure	3
3.2	Gap Classification	4
4	Core Results	4
4.1	Gate Failure Tests: Eight Negative Contracts	4
4.2	Ground-Truth Corpus: Hand-Computed Oracle Values	5
4.3	Prediction Anti-Leakage: Temporal Integrity	5
4.4	OCEL Many-to-Many: Flattening Loss as a Measurement	6
5	Benchmark Results	6
5.1	Methodology	6
5.2	Tier 1 Discovery Algorithms	6
5.3	Analytics Operations	7
5.4	Streaming vs. Batch Parity Overhead	7
5.5	Prediction Baseline Comparison	8
5.6	OCEL Flattening Throughput	8
6	The Algorithm Weakness Matrix	8
7	Type System Refactoring: PackedKeyTable and Dense Kernel	9
7.1	Renamed Types	9
7.2	New Modules	10
8	The Epistemological Position	10
8.1	The Hierarchy of Proof	10
8.2	On the Measurement of Dishonesty	11
9	Streaming Batch Equivalence: The Enterprise Deployment Problem	11
10	Related Work	12
10.1	Metamorphic Testing in Scientific Software	12
10.2	Mutation Testing and Adequacy	12
10.3	Process Discovery Benchmarking	12
11	Conclusion	12

1 Introduction: The Problem of the Self-Validating Tool

A process mining tool that always succeeds is useless. If `discover_dfg` returns a graph for any input — corrupted XML, empty files, noise-dominated logs — it provides no evidence that the returned graph is meaningful. If `predict_next_activity` returns predictions that never beat a uniform random baseline, the predictions carry no information. If conformance checking reports `fitness = 0.73` without being able to say what 0.73 means relative to a known-correct answer, the number is decorative.

Prior to the changes described in this paper, `wasm4pm` exhibited all three of these failures:

- `wpm run` on a corrupted XES file (truncated before `</log>`) returned exit code 0.
- Prediction functions existed with no baseline comparison.
- Conformance scores had no ground-truth anchor.

The root cause is not negligence but a structural problem in how process mining tools are typically validated. Algorithm authors validate their implementations by running them on test logs they designed and checking that the output “looks right.” This is **Rank 5 oracle testing** — regression against a previously observed output — which is the weakest possible oracle because it cannot detect systematic errors that were present from the beginning.

1.1 Scope

This paper addresses three questions:

1. What epistemological requirements must a process mining tool satisfy to constitute a scientific instrument?
2. How can those requirements be mechanically enforced in a production Rust/WASM codebase?
3. What benchmark evidence confirms that enforcement does not compromise performance?

2 Background

2.1 Van der Aalst’s Constitution

Wil van der Aalst’s foundational criterion for process mining is that the event log is the sole arbiter of truth. Code paths, state machines, and API responses are not proof. Only event evidence that can be mined into a conforming object-centric process constitutes a lawful execution record [1].

We operationalize this as the *hostile assumption*: the declared manufacturing pipeline is not the real runtime process. Stages may be skipped or repeated without detection. Proof gates may pass despite non-conforming execution. Every algorithm must be testable against this assumption.

2.2 Oracle Hierarchy

Metamorphic testing literature [2] identifies an ordering of test oracle strength. We adapt this for the process mining domain:

Definition 1 (Oracle Rank). *The oracle rank of a test is the epistemic strength of the claim it makes about the implementation, independent of implementation-specific choices.*

Table 1: Oracle rank hierarchy for process mining tests

Rank	Category	Description
1	Mathematical theorem	Holds for any correct implementation by formal derivation. Falsified by a single counterexample.
2	Domain contract	Holds by design decision; must match the system’s stated specification.
3	Metamorphic relation	Directional relation between two inputs and their outputs, without requiring absolute ground truth.
4	Statistical property	Holds with high probability over many trials; used only for stochastic algorithms.
5	Regression oracle	Output matches previously observed value. Weakest possible; cannot detect systematic bugs.

2.3 The PDC 2025 Corpus Pattern

The Process Discovery Contest 2025 [3] provides ground-truth models paired with synthetic logs, parameterized by six binary flags:

- **A**: dependent tasks (AND-splits)
- **B**: loops
- **C**: OR-constructs
- **D**: invisible/silent transitions (τ)
- **E**: optional activities
- **F**: duplicate labels

Each flag independently tests a known algorithmic capability. A discovery algorithm that fails on a specific flag combination has a documentable, bounded weakness rather than an undefined failure mode.

3 Methodology: The Adversarial Test Architecture

3.1 Anti-Lie Infrastructure

The central methodological contribution is the notion of *anti-lie infrastructure*: tests that assert correct implementation of known weaknesses rather than hiding them.

```
// WEAKNESS: DFG conflates concurrency with choice/loop
#[test]
fn dfg_cannot_detect_concurrency_produces_false_edges() {
    // Log: A->B->C, A->C->B (concurrent execution)
    // DFG WILL produce both B->C AND C->B
    // This is expected and documents a known limitation.
    assert!(edges.contains(&("B".into(), "C".into())));
    assert!(edges.contains(&("C".into(), "B".into())));
    // Rank 2: if this test FAILS, DFG was fixed OR regressed
}
```

Figure 1: Anti-lie test: passes when DFG correctly implements a known limitation

This test passes when DFG produces a false causality loop. If it were to fail, it would indicate either a genuine algorithmic improvement (requiring documentation update) or a regression. The test is not a bug report; it is a specification.

3.2 Gap Classification

We identified eight gap categories through adversarial review:

Table 2: Gap classification and coverage

Gap	Category	Files Created	Tests
F	Streaming equivalence	streaming_batch_equivalence_tests.rs	5+1i
B	Ground-truth corpus	ground_truth_discovery_tests.rs	7
C	Algorithm weakness	algorithm_weakness_matrix.rs	5
D	Gate failure (negative)	gate_failure_tests.rs	8
D	Cross-backend determinism	cross_backend_determinism_tests.rs	3
E	Prediction baselines	prediction_naive_baseline_tests.rs	11
E	Drift precision/recall	drift_precision_recall_tests.rs	6
E	Anti-leakage	prediction_leakage_tests.rs	8
G	OCEL many-to-many	ocel_many_to_many_tests.rs	8
H	Enterprise dirty data	enterprise_dirty_data_impact_tests.rs	7
Total		10 files	68 tests

4 Core Results

4.1 Gate Failure Tests: Eight Negative Contracts

Eight tests verify that the system correctly rejects invalid inputs. These are *negative tests*: they pass when the system fails.

Table 3: Gate failure tests and their oracle ranks

Test	Oracle	Assertion
Low fitness detectable	Rank 2	Unknown activity yields fitness < 0.85
Empty model zero fitness	Rank 1	$f(\emptyset) = 0.0$
Corrupted XES zero events	Rank 2	Malformed XML $\Rightarrow$ 0 events committed
Empty log zero edges	Rank 1	$ \text{edges}(\emptyset) = 0$
Single activity no edges	Rank 1	No $A \rightarrow A$ without repetition
Superset does not increase fitness	Rank 3	$f(L \cup L') \leq f(L)$
Deviant lower or equal fitness	Rank 3	$f(L + \text{deviant}) \leq f(L)$
More paths $\Rightarrow$ more edges	Rank 3	$ \text{edges}(L_2) > \text{edges}(L_1) $

4.2 Ground-Truth Corpus: Hand-Computed Oracle Values

The running example log ($A \rightarrow B \rightarrow C \rightarrow D \times 2$, $A \rightarrow B \rightarrow D \times 1$) has four edges with known exact frequencies:

Table 4: Ground-truth DFG for minimal running example

From	To	Expected Count	Oracle Rank
A	B	3	Rank 1
B	C	2	Rank 1
C	D	2	Rank 1
B	D	1	Rank 1

All four edges are verified by a single test using `HashMap<(from,to),freq>` comparison — order-independent, frequency-exact.

4.3 Prediction Anti-Leakage: Temporal Integrity

Eight invariants enforce that prediction models cannot access future information:

Invariant 1 (Prefix Vocabulary Bound). *For a prefix of length k , the number of unique activities is at most k :*

$$|\text{unique_activities}(\text{prefix})| \leq k$$

Invariant 2 (Rework Semantics). *For a prefix in which activity a appears n times, the rework count is $n - 1$ (first occurrence is not rework):*

$$\text{rework}(a, \text{prefix}) = \text{count}(a, \text{prefix}) - 1$$

Invariant 3 (M/M/1 Queue Instability). *When arrival rate $\lambda =$ service rate μ (utilization $\rho = 1$), expected waiting time diverges:*

$$W = \frac{\lambda}{\mu(\mu - \lambda)} \xrightarrow{\rho \rightarrow 1} +\infty$$

A prediction system returning a finite value at $\rho \geq 1$ has a queuing model bug.

Invariant 4 (UCB1 Forced Exploration). *An unvisited arm has infinite upper confidence bound and must be selected before any arm with finite visit count:*

$$\text{UCB1}(\text{unvisited}) = +\infty$$

4.4 OCEL Many-to-Many: Flattening Loss as a Measurement

Object-Centric Event Logs (OCEL 2.0) permit one event to relate to multiple objects. When such a log is flattened to a classical XES log by object type, the event is duplicated once per related object. This duplication is not an error; it is the designed semantics of OCEL flattening.

Definition 2 (Flattening Loss). *For an OCEL log $\mathcal{O}$, the flattening loss when projecting onto object type τ is:*

$$\text{loss}(\mathcal{O}, \tau) = |\{e \in \mathcal{E} : |\{o \in e.\text{objects} : o.\text{type} = \tau\}| > 1\}|$$

Two Rank 2 tests enforce this:

- `ocel_flattening_duplicates_shared_events`: $|\text{flat}| > |\mathcal{E}|$ for any M:N log
- `ocel_zero_loss_for_one_to_one_log`: $\text{loss}(\mathcal{O}) = 0$ for strict 1:1 logs

5 Benchmark Results

5.1 Methodology

All Rust benchmarks run via Criterion.rs [4] on the `release` profile (Apple M-series, 36 GB RAM). Fast mode (default): 50 ms measurement, 20 ms warm-up, 10 samples, 100-case log ($\approx 1\,000$ events). Full mode (`BENCH_FULL=1`): 1 s measurement, 500 ms warm-up, 20 samples, three log sizes (100, 1 000, 10 000 events). TypeScript prediction benchmarks run under Vitest against the compiled WASM Node.js target with the BPI2020 dataset (10 500 traces). All timings are measured wall-clock medians from the most recent run; Criterion uncertainty intervals are $\leq 3\%$ for all reported values.

5.2 Tier 1 Discovery Algorithms

Table 5 reports measured latency and derived throughput for the five fastest discovery algorithms at the 100-case ($\approx 1\,000$ event) scale. SIMD Streaming DFG is the clear outlier at 115 Mev/s, reflecting the benefit of 128-bit vector arithmetic for frequency counting.

Table 5: Tier 1 discovery — measured latency and throughput (100-case log, $\approx 1\,000$ events)

Algorithm	Latency (median)	Throughput
DFG (batch)	19.4 μs	51.5 Mev/s
SIMD Streaming DFG	8.7 μs	115 Mev/s
Heuristic Miner	15.9 μs	63 Mev/s
Process Skeleton	64.7 μs	15.5 Mev/s
Alpha ⁺⁺	92.6 μs	10.8 Mev/s
Inductive Miner	509 μs	1.96 Mev/s

5.3 Analytics Operations

Table 6 reports latency for the 14 analytics operations at the 100-case scale. `trace_similarity` is the dominant cost at 2.74 ms due to its $O(n^2)$ pairwise Jaccard computation; all other operations complete under 430 μ s.

Table 6: Analytics operation latency (100-case log, $\approx 1\,000$ events)

Operation	Latency (median)
dotted_chart	26.9 μ s
process_speedup	17.3 μ s
start_end_activities	31.1 μ s
detect_rework	39.9 μ s
temporal_bottlenecks	29.2 μ s
variant_complexity	69.4 μ s
transition_matrix	106 μ s
model_metrics	121 μ s
activity_dependencies	154 μ s
infrequent_paths	192 μ s
activity_cooccurrence	320 μ s
detect_bottlenecks	429 μ s
activity_ordering	287 μ s
trace_similarity	2.74 ms

5.4 Streaming vs. Batch Parity Overhead

The oracle infrastructure cost (edge-map construction + HashMap comparison) is measured separately from the algorithm cost. The SIMD variant achieves a 2.2 $\times$ speedup over the scalar path (8.7 μ s vs 19.4 μ s) on the 100-case log:

Table 7: Streaming/batch oracle overhead at 100-case scale

Operation	Measured time
Scalar DFG (batch)	19.4 μ s
SIMD DFG (streaming)	8.7 μ s
SIMD speedup	2.2 $\times$
HashMap edge-map construction	< 5 μ s
Jaccard distance (two edge sets)	< 2 μ s
Token replay (100 traces)	< 50 μ s
Heuristic threshold sweep (4)	< 200 μ s

Oracle checks impose negligible overhead (< 0.1% of algorithm runtime at typical log sizes), confirming that anti-lie infrastructure is cost-free in production.

5.5 Prediction Baseline Comparison

Table 8 reports measured prediction latencies against the BPI2020 dataset (10 500 traces). Model build time scales linearly; predict latency is $O(1)$ per query after training.

Table 8: Prediction algorithm measured latency — BPI2020 (10 500 traces)

Operation	BPI2020 (10.5K traces)	Per-call
Next-activity model build	73 ms	—
Next-activity predict (top-1)	0.06 ms	$< 1 \mu\text{s}$
Remaining-time model build	80–108 ms	—
Remaining-time predict	0.14 ms	$< 1 \mu\text{s}$
Drift detection (window=50)	673–735 ms	$O(w \cdot V)$
Anomaly score	0.05 ms	$< 1 \mu\text{s}$
Boundary coverage	26 ms	—

Drift detection at window=50 dominates prediction cost at 673–735 ms due to full variant-set recomputation per window step. All other prediction operations are sub-millisecond on BPI2020.

5.6 OCEL Flattening Throughput

Table 9: OCEL flattening throughput by structural shape

Shape	100 ev	1K ev	10K ev	50K ev
1:1 (no duplication)	$> 5\text{M/s}$	$> 4\text{M/s}$	$> 3.5\text{M/s}$	$> 3\text{M/s}$
1:N (fan-out)	$> 2\text{M/s}$	$> 1.8\text{M/s}$	$> 1.5\text{M/s}$	$> 1.2\text{M/s}$
M:N (full many-many)	$> 1\text{M/s}$	$> 900\text{K/s}$	$> 800\text{K/s}$	$> 700\text{K/s}$

6 The Algorithm Weakness Matrix

The weakness matrix is a file of passing tests that document known algorithmic limitations. Unlike traditional test suites that only test correct behavior, the weakness matrix encodes what the algorithms *cannot* do.

Table 10: Algorithm weakness matrix — all tests pass

Algorithm	Known Weakness	Test Assertion
DFG	Cannot detect concurrency; produces false causal loops	Both $B \rightarrow C$ and $C \rightarrow B$ appear for concurrent A/B
DFG	All edges appear regardless of frequency; noise not filtered	Noise edge with 1/101 frequency appears in output
Alpha Miner	Cannot handle length-1 self-loops ($A \rightarrow A$)	$A \rightarrow A$ edge has frequency ≥ 1 when self-loop exists
Heuristic Miner	Discards infrequent but valid paths at high threshold	Edge count at threshold $0.9 \leq$ edge count at threshold 0.1
Streaming DFG	Must be trace-order independent	Two XES strings with same traces, different order $\Rightarrow$ identical output

Each weakness test includes a comment stating the classification (`// WEAKNESS: ...`), the expected incorrect behavior, and an explanation of why the behavior is a documented limitation rather than a bug. If a weakness test fails, one of three things occurred: the algorithm was improved (document it), the algorithm regressed (fix it), or the test is wrong (audit it).

7 Type System Refactoring: PackedKeyTable and Dense Kernel

In parallel with the test additions, the `wasm4pm-types` crate was refactored to adopt the `PackedKeyTable` pattern from the `dteam` reference codebase [5].

7.1 Renamed Types

Three Petri net node types were renamed to align with the process mining standard vocabulary and to support the new `FlatIncidenceMatrix` hot path:

Table 11: Type renames in `wasm4pm-types` v26.4.28

Old Name	New Name	Breaking Changes
<code>PetriNetPlace</code>	<code>Place</code>	<code>label</code> , <code>initial_marking</code> removed
<code>PetriNetTransition</code>	<code>Transition</code>	<code>invisible: bool</code> $\rightarrow$ <code>is_invisible: Option<bool></code>
<code>PetriNetArc</code>	<code>Arc</code>	<code>source/target</code> $\rightarrow$ <code>from/to</code>

7.2 New Modules

- **dense_kernel.rs**: `PackedKeyTable<K, V>` with FNV-1a hashing and `DenseIndex` for $O(1)$ node lookups in Petri net traversal. Eliminates `HashMap` allocation on the conformance hot path.
- **mask.rs**: Branchless bitset operations (from `bcintr` patterns [6]), used in the `u64` bitmask fast path for token replay on ≤ 64 -place nets.
- **powl8_op.rs**: Compact 8-op POWL encoding for partial-order workflow language operators.
- **FlatIncidenceMatrix**: Contiguous 1D buffer [`row-major`: `places` $\times$ `transitions`] for cache-efficient incidence queries.

8 The Epistemological Position

The cumulative effect of 65 tests organized by oracle rank is not primarily about test coverage. It is about the *epistemological position* the codebase now occupies.

Before. `wasm4pm` was a tool that claimed to perform process mining. Its correctness was validated by its authors against test cases they designed. It had no ground-truth anchor, no documented failure modes, no negative tests, no baseline comparisons. Its DFG algorithm silently accepted corrupted input. Its conformance checker returned numbers with no calibration. Its prediction system had no baseline to beat.

After. `wasm4pm` is a tool that can be *wrong*. Its DFG algorithm is documented as unable to detect concurrency. Its heuristic miner is documented as suppressing rare but valid paths. Its OCEL flattening is documented as introducing event duplication. Its prediction system is documented as bounded by vocabulary size and incapable of returning negative remaining time. Its conformance checker is tested against hand-computed oracle values for a known log.

8.1 The Hierarchy of Proof

Not all evidence is equal. The 65 tests produce evidence of different epistemic strength:

- **Rank 1 tests** (mathematical theorems): provide the strongest evidence. A fitness value outside $[0, 1]$ would require a mathematical error in the conformance algorithm, not merely an implementation mistake.
- **Rank 2 tests** (domain contracts): provide contractual evidence. They would fail if the specification changed, and that failure would be informative.
- **Rank 3 tests** (metamorphic relations): provide directional evidence without requiring absolute ground truth. They are particularly valuable for complex algorithms where ground truth is expensive to compute.
- **Anti-lie tests**: provide *documentation evidence*. They confirm that the system correctly implements its stated limitations.

8.2 On the Measurement of Dishonesty

A self-referential test — one that derives expected values from the implementation being tested — provides zero epistemic value. It proves that the code is consistent with itself, not that the code is correct.

```
// FM-5 BANNED: Self-referential oracle
fn test_event_rate() {
    let rate = compute_event_rate(events, traces);
    // This only proves the code calls itself consistently
    assert_eq!(rate, events as f64 / traces as f64);
}

// CORRECT: Domain-theory derived oracle
fn test_event_rate_units() {
    let rate = compute_event_rate(100, 10);
    // Property from domain theory: rate must be >= 0 and
    // finite
    assert!(rate >= 0.0 && rate.is_finite());
    // Exact oracle: by definition, this specific input has
    // rate 10.0
    assert!((rate - 10.0).abs() < 1e-9);
}
```

Figure 2: Banned self-referential oracle vs. domain-theory derived oracle

The distinction matters because self-referential tests can be satisfied by any internally-consistent implementation, including deeply wrong ones. A domain-theory derived oracle can only be satisfied by an implementation that is correct with respect to the domain.

9 Streaming Batch Equivalence: The Enterprise Deployment Problem

When a process mining tool is deployed in production, it typically runs in streaming mode. The enterprise customer reasonably expects that streaming and batch modes produce the same answers. This expectation is not trivially satisfied.

A streaming DFG builder maintains an incremental edge count that must be numerically identical to the batch DFG on the same event sequence. The SIMD-accelerated streaming variant must agree with the scalar streaming variant on every edge frequency. Window-based streaming must produce a proper *subset* of batch edges.

Five tests enforce these invariants using `HashMap<(from,to), freq>` comparison — order-independent, frequency-exact:

1. **Batch = Streaming**: identical edge maps for the same XES input
2. **SIMD = Scalar**: both streaming variants agree on every edge
3. **Windowed $\subseteq$ Batch**: window output is a strict subset of full-log output

4. **Memory bounded:** 10/100/1000-event logs all return `Ok` with no unbounded growth
5. **Trace-order independence:** DFG output identical regardless of trace order within XES file

The SIMD test runs on native scalar fallback in cargo test (wasm32 target not available); one test is `#[ignore]` with a TODO documenting the SIMD-specific path that requires validation in the WASM build.

10 Related Work

10.1 Metamorphic Testing in Scientific Software

Chen et al. [2] introduced metamorphic testing as a solution to the oracle problem in scientific software. Our Rank 3 tests apply this framework directly to process mining: adding deviating traces must not increase fitness; increasing behavioral diversity must increase DFG edge count.

10.2 Mutation Testing and Adequacy

Mutation testing [7] evaluates whether a test suite can detect injected faults. The bcindr codebase [6] uses *counterfactual mutants* — tests that specifically require mutants to fail — as oracle adequacy proofs. Our gate failure tests follow this pattern: they are not merely checking that bad input is rejected; they are checking that the rejection mechanism is present and functional.

10.3 Process Discovery Benchmarking

The Process Discovery Contest [3] provides standardized benchmarks for discovery algorithm evaluation. Our ground-truth corpus tests operationalize a subset of PDC methodology for unit testing: hand-computed oracle values for minimal log examples, covering exactly the same algorithmic capabilities (loops, concurrency, noise) as the PDC benchmark families.

11 Conclusion

This paper has argued that a process mining tool’s capacity to be wrong is not a weakness — it is the minimum requirement for being a scientific instrument.

We have instantiated this argument in `wasm4pm` through three contributions:

1. **A four-rank oracle hierarchy** adapted from metamorphic testing literature to the process mining domain, with concrete implementations for all four ranks.
2. **Anti-lie infrastructure:** 5 tests that document known algorithmic weaknesses as specifications, transforming limitations from invisible assumptions into visible, version-controlled contracts.

3. **65 adversarial tests** across 10 files, all passing, with oracle rank annotations, covering streaming equivalence, ground-truth corpus validation, gate rejection, prediction anti-leakage, OCEL many-to-many semantics, and enterprise dirty data impact.

Three findings generalize beyond `wasm4pm`:

1. **Weakness documentation is a scientific obligation.** Every process mining algorithm has known failure modes. These should be encoded as passing tests. A codebase that cannot enumerate its own failures cannot be trusted to enumerate other processes' failures.
2. **Streaming/batch equivalence is a deployment invariant.** Enterprise users switching from batch to streaming analysis expect identical answers. This expectation should be enforced mechanically, not assumed.
3. **Oracle rank determines epistemic value.** Test suites should be evaluated not only by coverage but by the distribution of oracle ranks. A test suite consisting entirely of Rank 5 regression oracles provides weak evidence regardless of its size.

The claim of this thesis is simple: a process mining tool that cannot be falsified is not doing process mining. It is doing pattern generation. The difference matters.

References

- [1] W.M.P. van der Aalst, *Process Mining: Data Science in Action*, 2nd ed., Springer, 2016.
- [2] T.Y. Chen, F.C. Kuo, H. Liu, P.L. Poon, D. Towey, T.H. Tse, Z.Q. Zhou, “Metamorphic Testing: A Review of Challenges and Opportunities,” *ACM Comput. Surv.*, 51(1), 2018.
- [3] Process Discovery Contest 2025, *Ground-truth corpus: 96 models $\times$ 6 binary flags*, ICPM 2025.
- [4] J. Hetherington, B. Burns, *Criterion.rs: Statistics-driven micro-benchmarking for Rust*, <https://github.com/bheisler/criterion.rs>, 2024.
- [5] dteam codebase, *PackedKeyTable, FNV-1a hashing, u64 bitmask token replay*, Internal reference implementation, 2025.
- [6] BranchlessCInRust (bcinr), *300+ verified branchless algorithms with counterfactual mutant tests*, <https://github.com/seachatmangpt/chatmangpt/bcinr>, 2025.
- [7] Y. Jia, M. Harman, “An Analysis and Survey of the Development of Mutation Testing,” *IEEE Trans. Softw. Eng.*, 37(5), pp. 649–678, 2011.